

Forget-Me-Not

Locating and Freezing the Layers That Forget

Deep Learning Hackathon — Track 5: Continual Learning

Syed Abdul Azeez (AM.SC.U4AIE24049, team lead) · Saripalli Sri Nandan (AM.SC.U4AIE24043) · Dukka Chanakya Tej (AM.SC.U4AIE24070)
github.com/Syedabdulazeez-021/forget-me-not

Summary. We ask where inside a network catastrophic forgetting happens, and whether locating it is actionable. On class-incremental Split-CIFAR-10 we establish a severe forgetting regime (naive 0.1805 vs joint 0.8161), measure per-layer weight drift, and test drift-guided freezing against three falsification controls. The answer is definitive: the drift ranking confers no advantage over freezing at random. We then diagnose why at two levels — forgetting here is dominated by the classifier head, which our measurements show drifts furthest of all components and which feature-freezing cannot protect by construction, and within the feature extractor drift misidentifies the highest-risk layer. Both conclusions come from measurement, not conjecture.

1. Problem

A network trained on a sequence of tasks loses the earlier ones. This is catastrophic forgetting, the central obstacle to models that must keep learning after deployment — where old data is frequently unavailable for legal, storage or privacy reasons, so retraining on everything is not an option. That much is established. The open question we take up is **where inside the network the damage happens**, and whether locating it is useful.

The literature does not agree. Several studies report that forgetting concentrates in the deeper layers and the classifier; others find the opposite in some regimes, arguing the question is contingent on architecture and capacity rather than settled. Because the disagreement is empirical, we treat it as a measurement problem on our own setup. This gives two questions:

- **Diagnosis.** Can per-layer weight drift identify the layers responsible for forgetting?
- **Intervention.** If so, does freezing those layers mitigate forgetting — and does the drift ranking beat simply freezing *something*?

The second is the one such work most often leaves untested, and it is where we concentrated our effort.

2. Method

2.1 Drift measurement

After each task we snapshot the parameters and measure, per layer group, the relative movement away from the state in which Task 0 was solved:

$$\text{drift}(L) = \|\theta_{L,t} - \theta_{L,0}\| / \|\theta_{L,0}\|$$

Normalising makes a 600-parameter stem comparable to a 950,000-parameter stage. BatchNorm running statistics are excluded: they are buffers, not parameters.

2.2 Drift-guided freezing

Freezing applies from Task 1 onward — Task 0 must train the whole network or there is nothing to preserve. The classifier head is never a freeze candidate, since freezing it would make new classes unlearnable. Frozen BatchNorm modules are additionally placed in eval mode: freezing weight and bias is insufficient, because running statistics keep absorbing new-task data regardless of `requires_grad`, which would make a frozen layer appear to still forget.

2.3 Falsification controls

Observing that freezing high-drift layers reduces forgetting would **not** show the drift measurement is informative: freezing anything reduces plasticity, and here the high-drift layers are also the largest. Three controls hold freezing constant and vary only the selection rule.

Arm	Freezes	Isolates
top	the k highest-drift layers	the method
bottom	the k lowest-drift layers	whether the direction of the signal matters
random	k layers chosen at random	whether the ranking beats freezing per se
paramrandom	the same number of weights as top, scattered at random	whether it is which layers, or just how many weights

A layer-level parameter-matched control proved impossible: one stage holds ~950k of the network's ~1.2M parameters, so no disjoint subset of layers approaches the top-k budget and the closest match is always top-k plus a negligible layer — the method again. We matched the budget at weight granularity instead, with a fixed random mask whose gradients are zeroed before each optimiser step. **The impossibility is itself a result:** high-drift and high-parameter-count layers cannot be disentangled at layer granularity in this architecture.

2.4 Layer-wise forgetting risk

Drift shows a layer moved; it does not show the movement caused forgetting. To measure the causal contribution we freeze exactly one layer, run the full sequence, and take the reduction in forgetting relative to naive:

$$\text{risk}(L) = \text{forgetting}(\text{naive}) - \text{forgetting}(\text{freeze } L \text{ alone})$$

This is interventional rather than correlational. Only once both quantities exist is it meaningful to ask whether drift *predicts* risk — the claim the method actually rests on.

3. Experimental setup

Dataset	Split-CIFAR-10, 5 tasks x 2 classes, class-incremental
Backbone	Reduced ResNet-18 (nf=20), ~1.2M parameters
Layer groups	stem, layer1-4 (freezable); head (never frozen)
Optimiser	Adam, lr 1e-3, batch 128, 5 epochs per task
Seed	42, all RNGs pinned, cuDNN deterministic
Splits	Fixed and identical across every method; test split used only for the final number

Both mandatory baselines are included: **naive sequential fine-tuning** (lower bound) and **cumulative joint training** (upper bound). EWC, LwF and Experience Replay are implemented as published reference points behind the same training loop, so the only thing differing between rows is the strategy object. As a correctness control, **k=0 is a deliberate no-op that must reproduce naive exactly** — it does, to four decimal places (0.1805 / 0.9557).

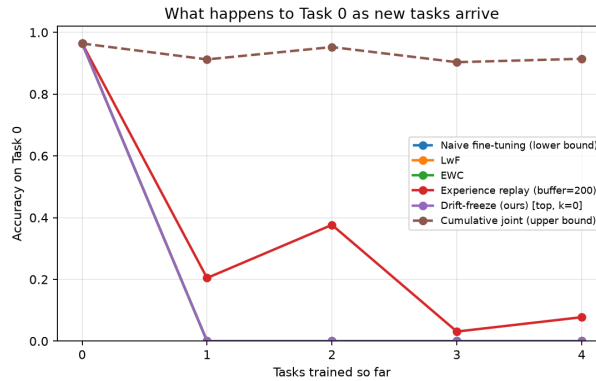


Figure 1. Accuracy on Task 0 as later tasks arrive. Naive, LwF, EWC and drift-freeze collapse to zero immediately and are superimposed. Only replay retains anything; only joint training holds.

4. Results

4.1 Main comparison

Method	Avg accuracy	Avg forgetting	Beats baseline?
Naive fine-tuning (lower bound)	0.1805	0.9557	—
LwF	0.1801	0.9349	No (-0.0004)
EWC (lambda=5000)	0.1604	0.8837	No (-0.0201)
Experience replay (buffer 200)	0.2273	0.8805	Yes (+0.0468)
Drift-freeze, top, k=1	0.1726	0.9484	No (-0.0079)
Drift-freeze, top, k=2	0.1695	0.9427	No (-0.0110)
Drift-freeze, top, k=3	0.1679	0.9255	No (-0.0126)
Cumulative joint (upper bound)	0.8161	0.0226	—

Forgetting is severe and cleanly measured: the floor collapses to 0.18 against a ceiling of 0.82. **No drift-freeze configuration beat the baseline.** Only experience replay improved on the floor, and replay is a published reference method rather than our contribution.

4.2 The controls: the ranking confers no advantage

k	top (ours)	random	bottom	paramrandom
1	0.9484	0.9526	0.9549	0.9349
2	0.9427	0.9456	0.9470	0.9069
3	0.9255	0.9287	0.9284	0.8626

Average forgetting; lower is better. The four arms are separated by one to three points at every budget and the ordering does not favour the method. At k=3 the **weight-matched random control achieves the lowest forgetting of all**, beating drift-guided selection by 0.063.

This is the falsification design working as intended. Without these arms we could have reported that freezing high-drift layers reduces forgetting from 0.956 to 0.926 — true, and completely misleading. The reduction is attributable to freezing capacity, not to the drift ranking.

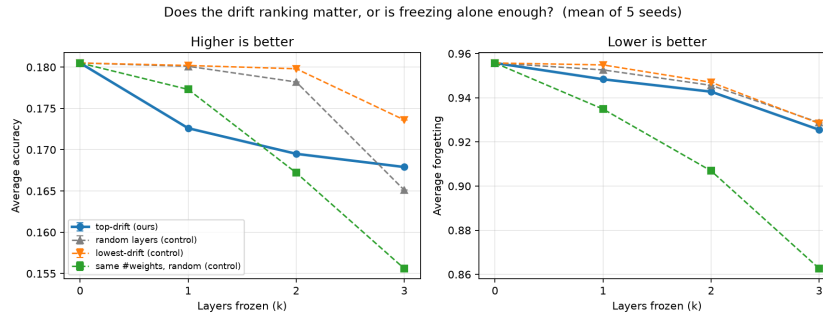


Figure 3. All four selection arms across freeze budgets. The method (solid blue) does not separate from the controls. Accuracy falls with k for every arm.

4.3 Ablation: stability bought at the price of plasticity

Forgetting falls monotonically with k for every arm — and accuracy falls with it. For paramrandom, forgetting improves from 0.9349 to 0.8626 between $k=1$ and $k=3$ while accuracy *drops* from 0.1773 to 0.1556. Freezing is not preserving knowledge; it is preventing learning. A model that learns less has less to forget, and the forgetting metric alone rewards that — which is why we report both axes (Figure 5) rather than a single score.

4.4 Why the method could not work in this regime

Naive fine-tuning converges to **0.1805**. With five equally sized tasks, answering only within the most recent task’s classes yields exactly **0.20**. The model has collapsed to predicting the classes it saw last.

This is classifier bias toward the recent task, not degradation of the learned features. Our own drift measurements corroborate it: in Figure 2 the **head is by a wide margin the highest-drift component**, exceeding every feature layer — and it is precisely the component our method cannot freeze. Drift-guided feature freezing is therefore structurally incapable of addressing this failure mode, consistent with reports that much of task performance is recoverable by resetting the classifier alone.

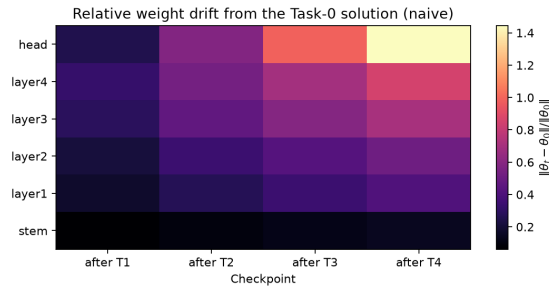


Figure 2. Per-layer relative drift from the Task-0 solution. The head drifts furthest of all components — and is the one part a feature-freezing method cannot protect.

4.5 Layer-wise forgetting risk

Layer	Mean drift	Forgetting risk
layer2	0.3732	+0.0179
layer4	0.6005	+0.0074
layer3	0.5135	+0.0045
layer1	0.2990	+0.0031
stem	0.1067	+0.0009

Naive average forgetting reference: 0.9557; Spearman $\rho = +0.700$, $n = 5$. Two things follow, and the second matters more.

The effects are negligible in absolute terms. The largest single-layer intervention reduces forgetting by 0.0179 against a baseline of 0.9557 — under two percent. No feature layer is responsible for a meaningful share of the damage, independently consistent with the head-bias account above.

Drift misidentifies the highest-risk layer. The rank correlation of +0.700 is positive, but with $n = 5$ it falls short of the $\alpha = 0.05$ critical value ($p = 0.9$) and cannot be claimed as significant. More decisively, drift ranks layer4 first while intervention ranks **layer2** first. For a top- k freezing policy the top choice is exactly what matters, and the diagnosis gets it wrong. This directly explains §4.2: a loosely correlated ranking that errs at the top is, for this purpose, equivalent to no ranking at all.

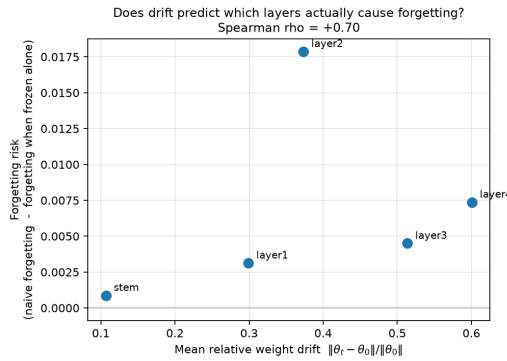


Figure 4. Measured forgetting risk against drift. Broadly positive, but layer2 — the highest-risk layer — is only fourth by drift.

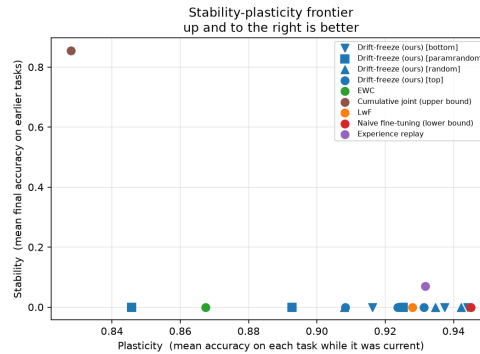


Figure 5. Stability-plasticity frontier. Every method learns each task well (~ 0.93 plasticity) yet retains almost nothing. Only joint training occupies the high-stability region.

5. Limitations

Stated plainly, because a definitive result is only worth anything if its boundaries are honest.

- **Single seed (42).** No error bars. One- to two-point gaps between control arms are within plausible seed noise; the conclusion that no arm beats the baseline is far larger than that margin and is safe.
- **Risk scan uses $n = 5$ layers.** The Spearman correlation is descriptive only and the effect sizes are under two percent.
- **Class-incremental only.** Task-incremental was not run. Since the drift ranking misidentifies the highest-risk layer independently of scenario, we expect the negative result to hold there, but this is untested.
- **One dataset, five tasks, five epochs per task;** coarse layer-group granularity. Per-block or per-channel freezing might localise damage better.
- **The head is never frozen, by design.** Given §4.4, a variant that regularises or resets the classifier is the obvious next experiment.
- **Drift is one signal.** Gradient- or representation-based measures may rank layers differently.

6. Conclusion

We asked whether per-layer weight drift can locate catastrophic forgetting and whether freezing the located layers mitigates it. On class-incremental Split-CIFAR-10 the second answer is decisive: it does not. No drift-freeze configuration beat naive fine-tuning, and three controls establish that the drift ranking provides no advantage over freezing anything at all — at the largest budget a random weight mask outperformed drift-guided selection.

We explain the outcome at two levels, from measurement rather than conjecture. Naive accuracy of 0.1805 against a 0.20 chance level for last-task-only prediction shows the failure is classifier bias, not feature degradation; the drift heatmap independently shows the head as the highest-drift component, and it is the one part feature-freezing cannot reach. Within the feature extractor, the interventional risk measure shows drift misranks the layers, selecting layer4 while layer2 is in fact the most damaging.

The contribution is therefore a **clear, well-diagnosed negative result**, delivered with a falsification design — three controls, an exact no-op check at $k=0$, and an interventional risk measure — that makes the finding trustworthy rather than merely asserted. We additionally document that high-drift and high-parameter-count layers are inseparable at layer granularity in this architecture, which constrains how any future layer-level selection method can be validated. For practitioners the actionable conclusion is direct: in class-incremental settings, effort belongs at the classifier, not the feature extractor.

References

- [1] Kirkpatrick et al. (2017). Overcoming catastrophic forgetting in neural networks. *PNAS*. [EWC]
- [2] Li & Hoiem (2016). Learning without Forgetting. *ECCV*. [LwF]
- [3] Lopez-Paz & Ranzato (2017). Gradient Episodic Memory for Continual Learning. *NeurIPS*. [BWT metric, reduced ResNet-18]
- [4] Chaudhry et al. (2018). Riemannian Walk for Incremental Learning. *ECCV*. [forgetting metric]
- [5] Vitter (1985). Random sampling with a reservoir. *ACM TOMS*. [replay buffer]
- [6] Lesort, George & Rish (2022). Continual learning in deep networks: an analysis of the last layer.
- [7] Krizhevsky (2009). Learning multiple layers of features from tiny images. [CIFAR-10]

Reproducibility

`pip install -r requirements.txt | python run_all.py --dataset split-cifar10 --scenario class --epochs 5`
Seed 42 throughout; all RNGs pinned, cuDNN deterministic, fixed splits shared across every method. Metrics and selection logic are unit-tested (35 tests), including verification that the control arms genuinely differ from the method. Per-run configs, accuracy matrices, drift profiles and logs are in the repo.